

Class03d2_Iris_2_sklearn

How to run: Double click Class03d2_Iris_2_sklearn.ipynb to open !

```
# from google.colab import drive
# drive.mount('/content/drive')
```

To use ANN for the Iris Classification using the **sklearn** deep learning framework.

```
import numpy as np
import pandas as pd
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
```

```
# header = None : means no header line in the front.
# Each column is given a name.
```

```
# data_raw = pd.read_csv('/content/drive/MyDrive/ELEC6604_Sept2022/Class4_folder/iris_dataset.csv', k
```

```
data_raw = pd.read_csv('iris_dataset.csv', header = None, sep = ',')
data_raw.columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
```

```
print('shape of data_raw is ', data_raw.shape)
```

```
shape of data_raw is (150, 5)
```

The main function that we will use

CASE 1: Use one output ; i.e. target output is 0,1 or 2

▶ # CASE 1: Use one output ; i.e. target output is 0,1 or 2

```
df1_X = data_raw[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]
```

The following line would map the names to an integer label:

```
df1_y = data_raw['class'].map({'Iris-setosa':0, 'Iris-versicolor':1, 'Iris-virginica':2})
```



```
# We load in 150 samples and the following line using "train_test_split" would split it  
# into 120 training data and 30 testing data (20% is for testing)
```

```
X1_train, X1_test, y1_train, y1_test = train_test_split(df1_X, df1_y, test_size = 0.2)
```

```
print(X1_train.shape)
```

```
# (120,4)
```

```
print(X1_test.shape)
```

```
# (30,4)
```

```
df1_y
```

```
(120, 4)
```

```
(30, 4)
```

```
0      0
```

```
1      0
```

```
2      0
```

```
3      0
```

```
4      0
```

```
..
```

```
145    2
```

```
146    2
```

```
147    2
```

```
148    2
```

```
149    2
```

```
Name: class, Length: 150, dtype: int64
```

Two hidden layers

The model is defined with two hidden layers of ten nodes below:

```
rgs1 = MLPRegressor(solver = 'lbfgs', alpha = 1e-10, hidden_layer_sizes = (10,10), random_state = 1, max_iter = 1000)
```

```
rgs1.fit(X1_train.values, y1_train.values)
```

Start the training using MLPRegressor

```
# MLPRegressor(activation='relu', alpha=1e-10, batch_size='auto', beta_1=0.9,
```

```
#     beta_2=0.999, early_stopping=False, epsilon=1e-08,
```

```
#     hidden_layer_sizes=(10, 10), learning_rate='constant',
```

```
#     learning_rate_init=0.001, max_iter=1000, momentum=0.9,
```

```
#     nesterovs_momentum=True, power_t=0.5, random_state=1, shuffle=True,
```

```
#     solver='lbfgs', tol=0.0001, validation_fraction=0.1, verbose=False,
```

```
#     warm_start=False)
```

```
"""
```

```
solver={'lbfgs', 'sgd', 'adam'}, default='adam'
```

The solver for weight optimization.

- 'lbfgs' is an optimizer in the family of quasi-Newton methods.
- 'sgd' refers to stochastic gradient descent.
- 'adam' refers to a stochastic gradient-based optimizer.

Note: The default solver 'adam' works pretty well on relatively large datasets

(with thousands of training samples or more) in terms of both training time and

validation score. For small datasets, however, 'lbfgs' can converge faster and perform better.

Notice that this ANN has ONE output. """

The hyperparameters

The optimization algorithm

Treat the classification
problem as a
regression problem



```
y1_ANN = rgs1.predict(X1_test)
print(y1_ANN.shape)
# (30,)
print(len(y1_ANN))
# 30
print('ANN output values (thirty of them): ')
print(y1_ANN)
print('ANN output values (after rounding): ')
y1_round = np.round(y1_ANN)
print(y1_round)
print('Desired output values (thirty of them): ')
print(y1_test.values)
```

Predict after the training, using the test values

Treat the classification problem as a regression problem
one output (the class index)

```
(30,)
30
ANN output values (thirty of them):
[ 1.97065068e+00  2.64090348e-02  2.00561372e+00  2.00192141e+00
 1.23677983e+00  2.22732972e-03  2.01825324e+00  3.84496195e-02
 9.82328410e-03  1.11631355e+00  1.10219059e+00  9.54279865e-01
-7.38365652e-03  2.01858852e+00  1.08758875e+00  1.98527128e+00
 3.16591092e-02  9.64725154e-01  1.03971802e+00  2.05080777e+00
-3.16093173e-02 -1.87998209e-02  2.01657352e+00 -7.47265060e-03
 3.62381802e-02  1.33930502e-03 -4.70477304e-03  7.49288325e-01
 1.78913690e+00  1.10910019e+00]
```

```
ANN output values (after rounding):
[ 2.  0.  2.  2.  1.  0.  2.  0.  0.  1.  1.  1. -0.  2.  1.  2.  0.  1.
 1.  2. -0. -0.  2. -0.  0.  0. -0.  1.  2.  1.]
```

```
Desired output values (thirty of them):
[2 0 2 2 1 0 2 0 0 1 1 1 0 1 1 2 0 1 1 2 0 0 2 0 0 0 0 1 2 1]
```

Note that simple rounding of the ANN output values are used !

```
[ ] print('Differences between the ANN and Desired outputs: ')
    print(np.abs(y1_round - y1_test.values))
    print('Number of output cases: ')
    print(len(y1_round))

    print('Next is to calculate the accuracy in percentage: ')
    acc1 = (1 - np.round(sum(np.abs(y1_round - y1_test.values)/len(y1_round)),4))*100
    print('The Accuracy is:', acc1, '%')
```

Differences between the ANN and Desired outputs:

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.]
```

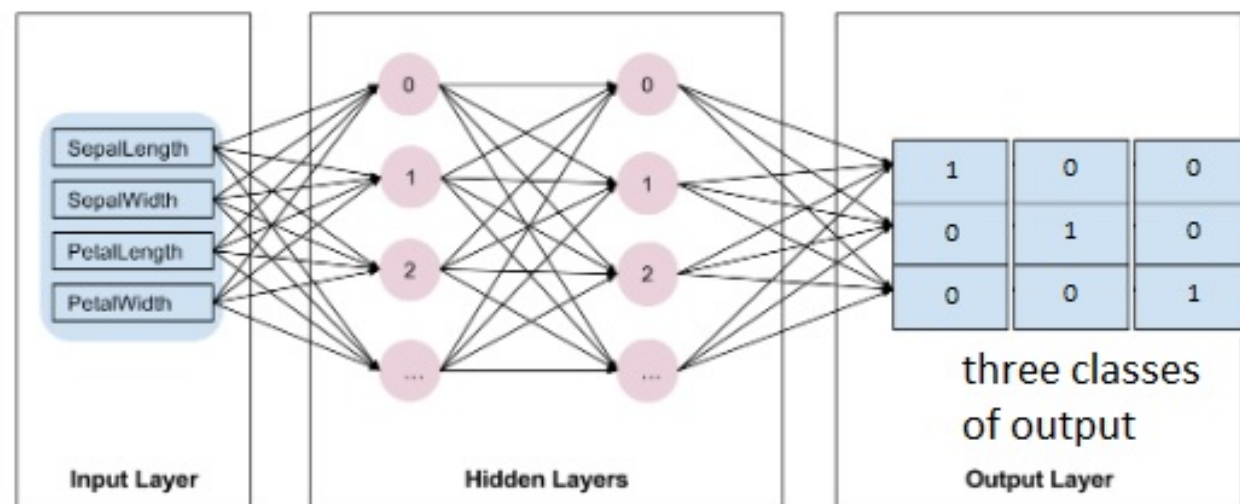
Number of output cases:

30

Next is to calculate the accuracy in percentage:

The Accuracy is: 96.67 %

CASE 2: Use three outputs



In the example of CASE 2, we define three outputs for the ANN. Note that rounding is also used for the three ANN outputs.



CASE 2: Use three outputs

```
# The following creates three more columns, and use three digits to represent each class
df2 = data_raw.copy()
df2['class1'] = df2['class'].map({'Iris-setosa':1})
df2['class2'] = df2['class'].map({'Iris-versicolor':1})
df2['class3'] = df2['class'].map({'Iris-virginica':1})
df2 = df2.fillna(0)

print('df2 = ',df2)
```

```
df2_X = df2[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]
df2_y = df2[['class1', 'class2', 'class3']]
```

```
X2_train, X2_test, y2_train, y2_test = train_test_split(df2_X, df2_y, test_size = 0.2)
```

```
rgs2 = MLPRegressor(solver = 'lbfgs', alpha = 1e-10, hidden_layer_sizes = (10,10), random_state = 1, max_iter = 1000)
```

```
rgs2.fit(X2_train.values, y2_train.values) ← Start the training
```

```
y2_ANN = rgs2.predict(X2_test) ← Predict after the training, using  
the test values
```

```
y2_ANN_round = np.round(y2_ANN)
```

```
# Note that a simple "round off" is used to find the nearest match to the output class.
```

```
error_output = (y2_ANN_round - y2_test).values
```

Subtraction

```
print('*** error_output is ', error_output)
```

```
# Number of test cases
```

```
len(error_output)
```



```

, df2 =      sepal_length  sepal_width  petal_length  petal_width      class \
0          5.1           3.5           1.4           0.2      Iris-setosa
1          4.9           3.0           1.4           0.2      Iris-setosa
2          4.7           3.2           1.3           0.2      Iris-setosa
3          4.6           3.1           1.5           0.2      Iris-setosa
4          5.0           3.6           1.4           0.2      Iris-setosa
..          ...           ...           ...           ...           ...
145         6.7           3.0           5.2           2.3      Iris-virginica
146         6.3           2.5           5.0           1.9      Iris-virginica
147         6.5           3.0           5.2           2.0      Iris-virginica
148         6.2           3.4           5.4           2.3      Iris-virginica
149         5.9           3.0           5.1           1.8      Iris-virginica

```

	class1	class2	class3
0	1.0	0.0	0.0
1	1.0	0.0	0.0
2	1.0	0.0	0.0
3	1.0	0.0	0.0
4	1.0	0.0	0.0
..	...	...	...
145	0.0	0.0	1.0
146	0.0	0.0	1.0
147	0.0	0.0	1.0
148	0.0	0.0	1.0
149	0.0	0.0	1.0

```
[150 rows x 8 columns]
*** error_output is [[ 0.  0. -0.]
 [ 0.  0. -0.]
 [ 0.  0.  0.]
 [ 0.  0.  0.]
 [ 0.  0.  0.]
```

Three output regression problem

```
[ 0.  0. -0.]  
[ 0. -1.  1.]  
[ 0.  0.  0.]  
[ 0.  0. -0.]  
[ 0.  0.  0.]  
[ 0.  0.  0.]  
[ 0.  0. -0.]  
[ 0.  0.  0.]  
[ 0.  0. -0.]  
[ 0.  0. -0.]
```

```
/usr/local/lib/python3.7/dist-packages/sklearn/base.py:444: UserWarning: X has feature names,  
f"X has feature names, but {self.__class__.__name__} was fitted without"  
30
```

```
for i in range(0, len(error_output)):  
    if (error_output[i][0] != 0 or error_output[i][1] != 0 or error_output[i][2] != 0):  
        print('Case ', i, ' is a mistake !')
```

```
Case 21 is a mistake !
```